→ UNION clause: used to combine results of two or more SELECT statement without returning any duplicate value

```
SELECT   ID, NAME, AMOUNT, DATE
FROM     CUSTOMERS
LEFT     JOIN   ORDERS
ON       CUSTOMERS. ID = ORDERS . CUSTOMER_ ID

UNION

SELECT     "    "     "    "
 FROM      "
RIGHT  JOIN ORDERS
   "       "     "     "    "
```

Output

| ID | Name | Amount | date |
|---|---|---|---|
| 1 | Ramesh | NULL | NULL |
| 2 | Khilan | 1560 | Dec |
| 3 | kaushik | 3000 | " |
| 3 | kaushik | 1500 | " |
| 4 | chiatali | 2060 | " |
| 5 | Hardik | NULL | NULL |
| 6 | Komal | NULL | NULL |
| 7 | Muffy | NULL | NULL |

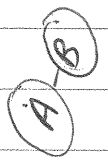
UNION ALL: used to combine results of two select statement including duplicate rows.

(result would be as full join)

→ Two operators similar to UNION clause.

(i) Intersect clause (common rows of both select statement)

```
SELECT ID, NAME, AMOUNT, DATE
FROM CUSTOMERS
LEFT JOIN ORDERS
ON CUSTOMER.ID = ORDERS.CUSTOMER_ID
INTERSECT
    SELECT  "   . . . . . .
    FROM    "   . . . . . .
    RIGHT JOIN ORDERS
    ON      "   " . . . . . .
```




| output | | | Date |
|---|---|---|---|
| 3 | kaushik | 3000 | " |
| 3 | " | 150 | " |
| 2 | Ramesh | 1560 | " |
| 4 | kaushik | 2060 | " |

(ii) EXCEPT clause same query only except in place of intersect used to combine two select statement and return rows first select statements that are not available in second select statement. (means returns rows that are not available in second SELECT statement).

| | | Amount | Date |
|---|---|---|---|
| 1 | Ramesh | NULL | NULL |
| 5 | Hardik | " | " |
| 6 | Komal | " | " |
| 7 | muffy | " | " |

→ Alias clause : rename a table or column temporarily by giving another name known as Alias
Actual name doesnot get change.

usage of table alias

alias

```
SELECT  C.ID, C.NAME, C.AGE, O.AMOUNT
FROM    CUSTOMER AS C , ORDERS AS O
WHERE   C. ID = O. Customer_ID
```

usage of column alias

```
SELECT  ID AS CUSTOMER_ID , NAME AS  CUSTOMER_NAME
FROM CUSTOMERS
WHERE  SALARY  IS  NOT NULL;
```

| ID | Name | age | amount |
|---|---|---|---|
| 3 | kaushik | 23 | 3000 |
| 3 | " | 23 | 1500 |
| 2 | kerilar | 25 | 1560 |
| 4 | Chaitali | 25 | 2060 |

| customer_id | customer_name |
|---|---|
| 1 | Ramesh |
| 2 | khilan |
| 3 | kaushik |
| 4 | Chaitali |
| 5 | Hardik |
| 6 | Komal |
| 7 | muffay |

→ Indexes - are special lookup tables that database search engine can use to speed up data retrieval. (like back of book) Indexes can be created or dropped with no effect on data.

Syntax:

| | |
|---|---|
| Create index | CREATE INDEX index_name ON table_name; |
| Single column index | CREATE INDEX index_name ON table_name (column_name); |
| Unique index | CREATE UNIQUE INDEX index_name on table_name (column_name); |
| Composite index | CREATE INDEX index_name on table_name ( column1, column2); |

Implicit index: indexes that are automatically created by database server when an object is created. Indexes are automatically created for primary key constraints and unique constraints.

DROP INDEX command : DROP INDEX index_name;

When should index be avoided.

(I) should not be made on small tables
(II) table that have frequent, large update or insert oper
(III) ~~used~~ should not be made on columns that contain a high no. of NULL values
(IV) column that are frequently manipulated should not be indexed.

ex: ALTER TABLE CUSTOMERS ADD SEX char(1);

this will add this to existing table →






→ ALTER table command: used to add, delete or modify column in an existing table.

syntax

ALTER table add new column → ALTER TABLE table_name ADD column_name datatype;

Drop column — ALTER TABLE table_name DROP COLUMN column_name;

data type — ALTER TABLE table_name MODIFY COLUMN column_name datatype;

NOT NULL — ALTER TABLE table_name MODIFY column_name datatype NOT NULL;

~~ADD PRIMARY KEY~~ → basic syntax to ADD UNIQUE CONSTRAINT

ALTER TABLE table_name
ADD ~~constraint~~ CONSTRAINT MyuniqueConstraint UNIQUE (col1, col2, ...);

to ADD CHECK CONSTRAINT

ALTER TABLE table_name
ADD CONSTRAINT myuniqueConstraint CHECK (cond$^n$);

to ADD PRIMARY KEY

ALTER TABLE table_name
ADD CONSTRAINT myPrimarykey PRIMARY KEY (col1, col2, ...);

to DROP constraint

ALTER TABLE table_name
DROP CONSTRAINT myuniqueconstraint; / myPrimarykey;

If to drop primary key

→ TRUNCATE TABLE : used to delete complete data from existing table.

```
TRUNCATE TABLE CUSTOMERS;
```

→ VIEW: (are a kind of virtual table) Contains all rows of a table or select rows from a table. A view can be created from one or many tables which depends on view query

```
CREATE VIEW view_name AS
SELECT col1, col2 ...
FROM table_name
WHERE [cond^];
```

VIEW can be created from single table, multiple table, or another view.

| ID | NAME | AGE | ADDRESS | SALARY |
|---|---|---|---|---|
| .. | | | | |
| ... | | | | |

Previous table

```
CREATE VIEW CUSTOMER_VIEW AS
SELECT AGE, NAME
FROM CUSTOMERS;
```

Output:

| age | name |
|---|---|
| 32 | Ramesh |
| 25 | khilan |
| 23 | kaushik |
| 25 | Chaitali |
| 27 | Hardik |
| 22 | komal |
| 24 | muffy |

with check option

```
CREATE  VIEW  CUSTOMER_VIEW  AS
SELECT  name, age
FROM    CUSTOMERS
WHERE   age  IS  NOT NULL
WITH    CHECK OPTION
```

This case would deny entry of any NULL values in view's age column, because view is defined by data that doesnot have NULL value in AGE column.

```
UPDATE  CUSTOMER_VIEW
 SET    AGE = 35
WHERE   name = 'Ramesh';
```

output:

| name | age |
|---|---|
| Ramesh | 35 |
| ⋮ | |
| ⋮ | |
| ⋮ | |
| ⋮ | |

We cannot insert a row in customer_view bcz we have not included all NOT NULL column in this view, otherwise you can insert a row

```
DELETE  FROM  CUSTOMER_VIEW
 WHERE  age = 22;
```

↑ all name that contain age = 22 will get deleted

```
DROP  VIEW  CUSTOMER_VIEW
```

↑ it will drop customerview.

→ updating a view.

(i) Select clause may not contain keyword distinct, summary func^n, set func^n, set operators, ORDER BY

(ii) FROM clause may not contain multiple tables.

(iii) WHERE clause may not contain subqueries.

(iv) query may not contain GROUP BY or HAVING.

(v) calculated columns may not be updated

(vi) All NOT NULL columns from base table must be included in view in order for INSERT query to func^n.

→ Having clause : enables you to specify cond^n that filters which group results appear in final result.

WHERE clause places cond^n on selected col^n, whereas HAVING " " " on groups created by GROUP BY clause.

```
SELECT   col1, col2
FROM     table1, table2
WHERE    [cond^n]
GROUP BY col1, col2
HAVING   [cond^n]
ORDER BY col1, col2
```

```
select *
FROM   CUSTOMERS
GROUP BY age
HAVING COUNT(age) >= 2;
```

Output :

| ID | NAME | AGE | ADDRESS | SALARY |
|---|---|---|---|---|
| 2 | khilan | 25 | Delhi | 1500.00 |

To read more about transaction control
www.tutorialspoint.com/sql/sql-transactions.htm



Transactions : unit of work that is performed against a database.

A transaction is propogation of one or more changes to database.

ex: if you are creating a record, updating, deleting a record, then you are performing a transaction on table.

Properties

(i) Atomicity: ensures all operation within work unit are completed succesfully, if not, transaction is aborted at point of failure, previous operations are rolled back to their former state.

(ii) Consistency: ensures that database properly changes states upon a succesfully committed transaction

(iii) Isolation: enables transactions to operate independently of and transparent to each other.

(iv) Durability: ensures that result or effect of committed transaction persists in case of system failure

Transaction control

(i) COMMIT : to save changes
(ii) ROLLBACK : to rollback changes
(iii) SAVEPOINT : creates points within groups of transactions in which to ROLLBACK.
(iv) SET TRANSACTION : Place a name on transaction.

# DBMS

Database (DB) - collection of related data.

Datawarehouse (DW) - database is very huge and contains historical data ( ex: past few years company details )

GIS: geographical information system database. ( NASA uses)

DBMS - set of programs which are used to define, construct & manipulate the database.
↑ software

DB + DBMS ⟶ called DBS

→ Database models.

(i) Highlevel modelling or conceptual :

(ii) Representational or implementation. generally tables, relations

(iii) LL or physical data models : generally implement tables how to store them in memory device how to retrieve it etc.